

Product Definition: API Reliability & Testing Platform

1. The problem

Developers can test an API manually, but problems often appear **after deployment**:

- Endpoint goes down
- Response becomes slow
- API returns the wrong status code
- Response structure changes
- Required fields disappear
- Authentication breaks
- An external API starts failing
- A new deployment silently breaks an existing endpoint

Existing monitoring tools often tell you:

"Your endpoint is down."

Our platform should go further:

"Your endpoint is technically up, but its response has changed and your `user_id` field is now missing."

That's the distinction we want.

2. Target users

Primary

- Backend developers
- Full-stack developers
- Small engineering teams
- SaaS developers

Secondary

- QA engineers
- DevOps engineers
- Agencies managing client APIs

We should **not** target enterprises initially. That adds unnecessary complexity.

3. Core product

A developer adds an API endpoint and defines what "healthy" means.

Example:

GET `https://api.example.com/users/123`

Expected:
Status: 200
Response time: < 500ms

Body:
user.id → required
user.email → required
user.name → string

The platform repeatedly tests it.

If something changes:

⚠ API CONTRACT FAILURE

GET /users/123

Expected:
user.email → string

Received:
user.email → null

First detected:
02:14 AM

That is much more valuable than simply checking whether the server returned 200.

4. MVP

The first version should have **5 core capabilities**:

A. API Monitoring

- HTTP/HTTPS endpoints
- GET, POST, PUT, PATCH, DELETE
- Status-code checks
- Response-time checks
- Request headers
- Request body

B. Response Assertions

Users can define:

- Status code
- Response time
- JSON field exists
- JSON field type
- JSON field value
- Header exists/value

C. Scheduled Testing

For example:

Every 1 minute
Every 5 minutes
Every 15 minutes
Every hour

D. Failure Detection

Detect:

- Downtime
- Timeout
- Unexpected status
- Slow response
- Assertion failure
- Contract/schema change

E. Alerts

Start with:

- Email
- Webhook

Don't build Slack, Discord, SMS, WhatsApp, etc. yet.

5. Dashboard

The main dashboard should answer three questions immediately:

Is everything healthy?

What is failing?

Has reliability improved or deteriorated?

Example:

API Health

98.7%

12 Monitors

10 Healthy

1 Degraded

1 Failing

Recent incidents

✖ POST /orders
Contract failure
2 minutes ago

⚠ GET /products
Response time increased
18 minutes ago

6. What makes it different

This is important.

We shouldn't compete on:

"We monitor APIs."

We compete on:

"We continuously verify that your API behaves the way it is supposed to."

That's **API reliability** + **contract monitoring**, not just uptime monitoring.

7. What we deliberately exclude from MVP

No:

- API client like Postman
- API documentation generator
- Load testing
- Full CI/CD platform
- AI agent
- Kubernetes monitoring
- Server monitoring
- Mobile app
- Team billing
- Complex RBAC
- Enterprise SSO

Those can come later.

8. MVP success test

I'd consider the MVP successful if a developer can:

Add an endpoint → define expected behavior → schedule tests → receive a failure alert → inspect exactly what changed.